Homework 4 PCA

Task 1: Read answers.txt and convert the letters back into numerical values *{1, 2, 3, 4}*

```python
# task 1
with open("../作业/answers.txt", "r") as f:
    lines = [line.strip() for line in f.readlines() if line.strip()]

replace_map = {"A": 1, "B": 2, "C": 3, "D": 4}
matrix = []
for line in lines:
    num_line = [replace_map[char] for char in line]
    matrix.append(num_line)

# 构造X
X = np.array(matrix)
X
```

✓ 0.0s Python

```
array([[1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1],
       [4, 4, 4, 4, 4, 4, 4, 4, 4, 4, 4, 4, 4, 4, 4, 4, 4, 4, 4, 4],
       [3, 3, 3, 4, 3, 3, 4, 4, 4, 4, 3, 4, 4, 3, 4, 3, 4, 4, 3, 3],
       [3, 2, 3, 3, 2, 3, 3, 3, 3, 4, 4, 3, 2, 2, 3, 3, 3, 3, 3, 2],
       [4, 3, 4, 4, 3, 3, 4, 3, 4, 3, 4, 4, 3, 4, 4, 4, 3, 3, 3, 4],
       [1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1],
       [1, 2, 1, 2, 1, 2, 1, 2, 2, 1, 2, 2, 2, 2, 2, 1, 2, 1, 1, 1],
```

Task 2: Determine the dimensions of the data matrix from the text file

```python
# task 2
sample_num , feature_num = X.shape
print(f"number of samoles:{sample_num}\nnumber of features:{feature_num
```

✓ 0.0s Python

```
number of samoles:25
number of features:20
```

Task 3: Apply PCA to the numerical matrix and keep only the first 2 principal components

```python
# task 3
pca = PCA(n_components = 2)
data = pca.fit_transform(X)

```

Python

Task 4: Reconstruct the original data matrix from the PCA representation.

```python
# task 4: reconstruction
data_reconstruct = pca.inverse_transform(data)

```

✓ 0.0s Python

Task 5: Round the reconstructed entries to the nearest value in $\{1, 2, 3, 4\}$, and clip values if necessary so that all reconstructed entries stay within this range

```python
# task 5
# 转回字符
data_rounded = np.clip(np.round(data_reconstruct), a_min = 1, a_max = 4
res = []
reconstruction_map = {1:"A", 2:"B", 3:"C", 4:"D"}
for row in data_rounded:
    char_row = [reconstruction_map[i] for i in row]
    res.append(char_row)
print(res)
```

✓ 0.0s Python

```
[['A', 'A', 'A', 'A', 'A', 'A', 'A', 'A', 'A', 'A', 'A', 'A', 'A', 'A', 'A',
```

Task 6: Compute the reconstruction accuracy:

```python
# task 6: Accuracy
total = 20 * 25
bias = 0
for i in range(len(res)):
    for j in range(len(res[0])):
        if res[i][j] != lines[i][j]:
            bias += 1
print(f"Accuracy:{(total - bias) / total}")
```

✓ 0.0s

```
Accuracy:0.782
```

Task 7: Compare the storage requirement of the original data with the storage requirement of the PCA representation.

```python
# task 7: storage requirement
original_storage = 20 * 25
pca_storage = 20 * 2 + 2 * 25 + 25  # 降维后矩阵 + 主成分 + 均值
print(f"original storage requirement:{original_storage}")
print(f"pca storage requirement: {pca_storage}")
```

✓ 0.0s Pyth

```
original storage requirement:500
pca storage requirement: 115
```